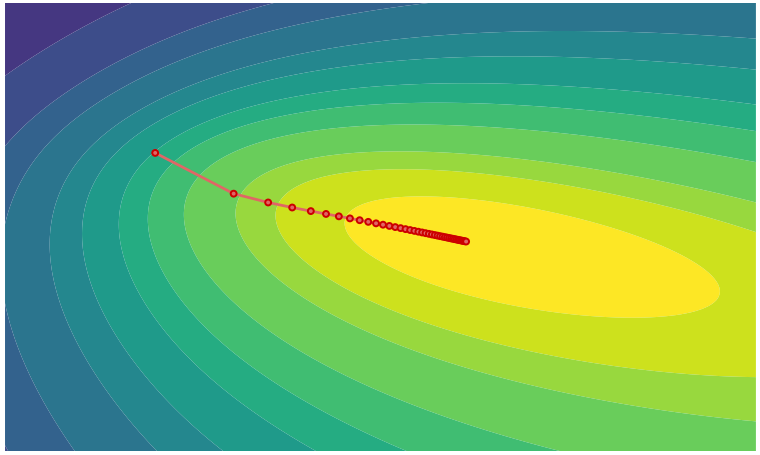
UNIT 01 · CHAPTER 3

Optimization

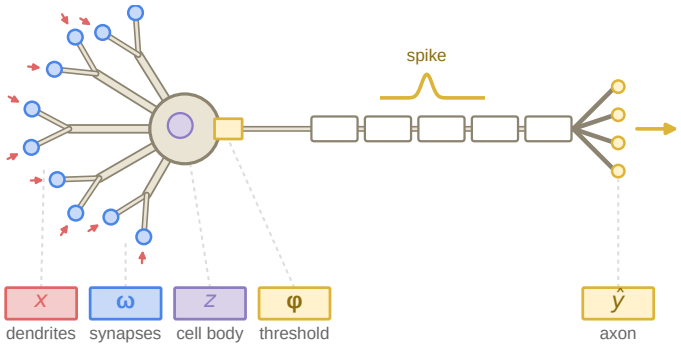
CSE 10124 · Building ChatGPT
William Theisen · University of Notre Dame

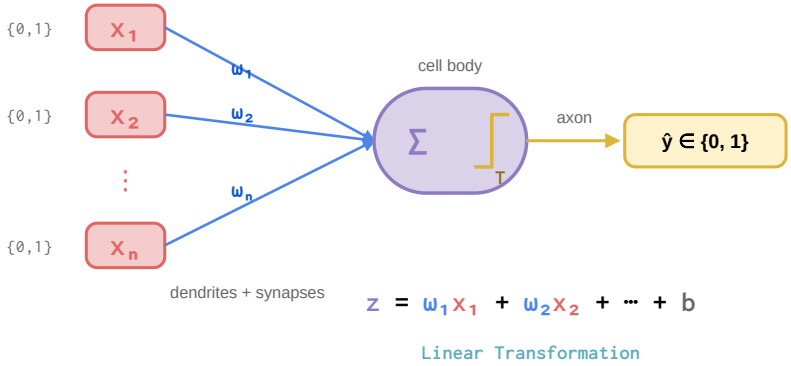


McCulloch–Pitts Artificial Neuron (1943)

What a **neural network** is trying to model is the way a human brain works, by combining and chaining together **artificial neurons** in **layers**.

- **Layer**: a group of neurons that process inputs in parallel and pass their outputs to another layer.
- **Neuron**: a computational unit with a set of **parameters** (weights and biases) that creates a weighted sum of the input.





Loss Functions

Loss: a single numerical measure of how well a model's prediction matches the expected answer.

Loss Function: a function that calculates our loss. **`Cross-Entropy`** is the loss function we'll use primarily in this course.

$$\mathcal{L}_{ce} = -\frac{1}{N}\left(\sum_{i=1}^{N} y_i \cdot \log(\hat{y}_i)\right)$$

Cross-Entropy: the loss function used to train LLMs, and one of the most common loss functions. Heavily based on the principle of "entropy" as used in Information Theory, it's a measure of how surprising our answer is relative to our ground truth, based on the entropy formula $-\Sigma\ p(x)\ \log_2\ p(x)$. Mostly outside the scope of the class, but it's really not *that* complicated.

Mathematical Formulation of a Neural Network

We can write our **loss** as a function of our **input** and **parameters**.

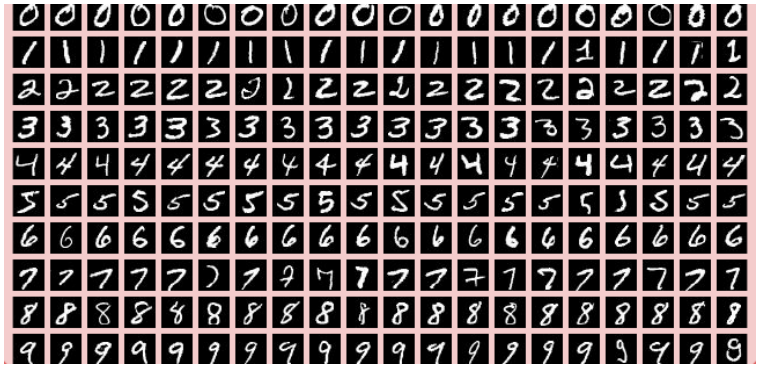


x: <1, 784> W: <10, 784> b: <1, 10> y: <1, 10>, one-hot

$$3.019 = \mathcal{L}_{ce} = -\frac{1}{N}\left(\sum_{i=1}^{N} y_i \cdot \log(\text{softmax}(x W^T + b))\right)$$

Formulating a **neural network** mathematically in this manner is **critical to actually training one**. Wednesday we'll dig more into this, and it will be the "mathiest" lecture in this course.

Multiple Layers and Batches

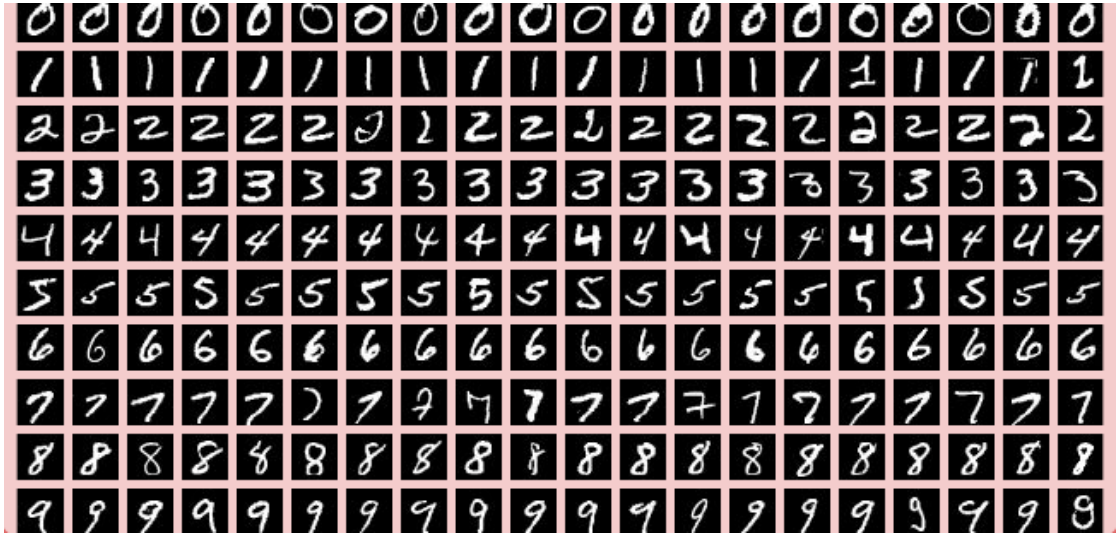


$$Z_1 \underset{\langle 64,\ 256\rangle}{} = X_{\langle 64,\ 784\rangle} \ @\ W_1^{T}{}_{\langle 784,\ 256\rangle} + b_1{}_{\langle 1,\ 256\rangle}$$

$$Z_2 \underset{\langle 64,\ 10\rangle}{} = Z_1{}_{\langle 64,\ 256\rangle} \ @\ W_2^{T}{}_{\langle 256,\ 10\rangle} + b_2{}_{\langle 1,\ 10\rangle}$$

$$\mathcal{L}ce = CE\left(Z_2{}_{\langle 64,\ 10\rangle},\ y_{\langle 64,\ 10\rangle}\right)$$

Multiple Layers and Batches

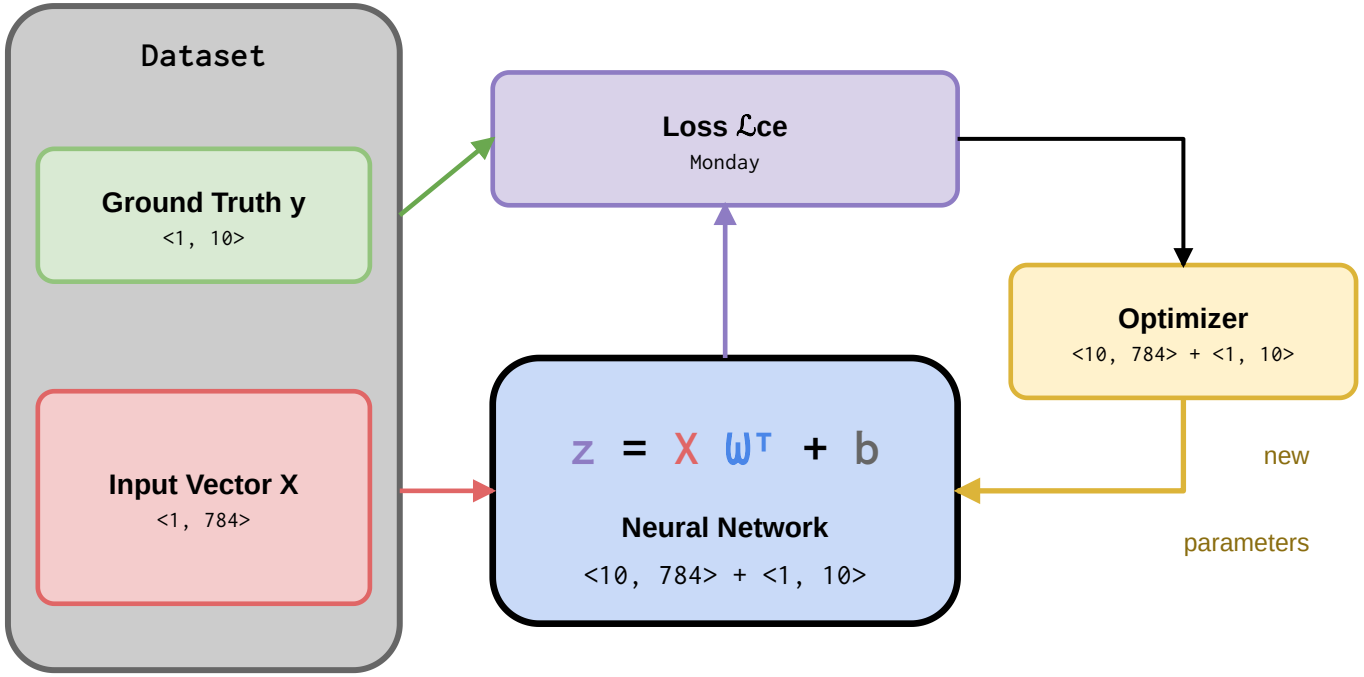


X: <64, 784> W_1^T: <784, 256> b_1: <1, 256> W_2^T: <256, 10> b_2: <1, 10> y: <64, 10>, the ground truth vectors

$$\mathcal{L}_{ce} = -\frac{1}{N}\left(\sum_{i=1}^{N} y_i \cdot \log(\text{softmax}((X W_1^T + b_1) W_2^T + b_2))\right)$$

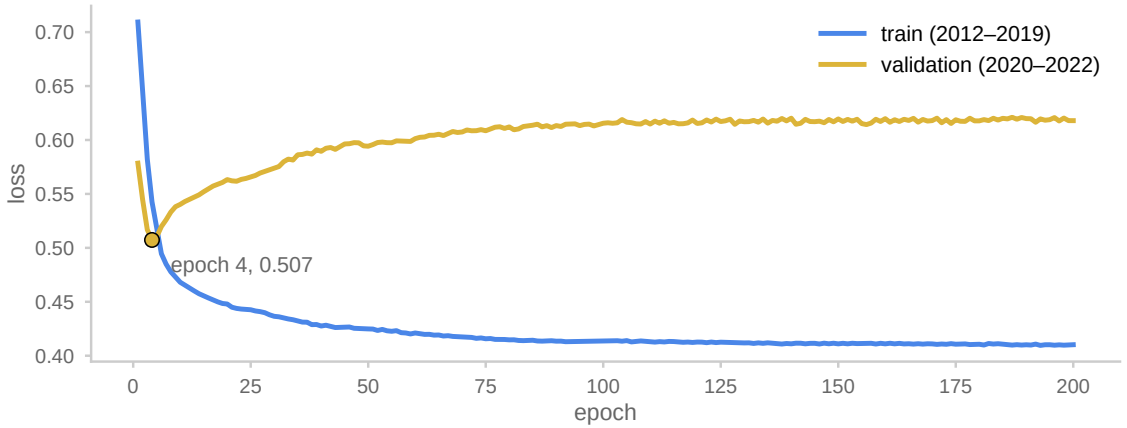
Training a Model

Training a model lets it practice over and over on the same set of data, ideally **improving its performance** over time! It does this by updating its **parameters**.



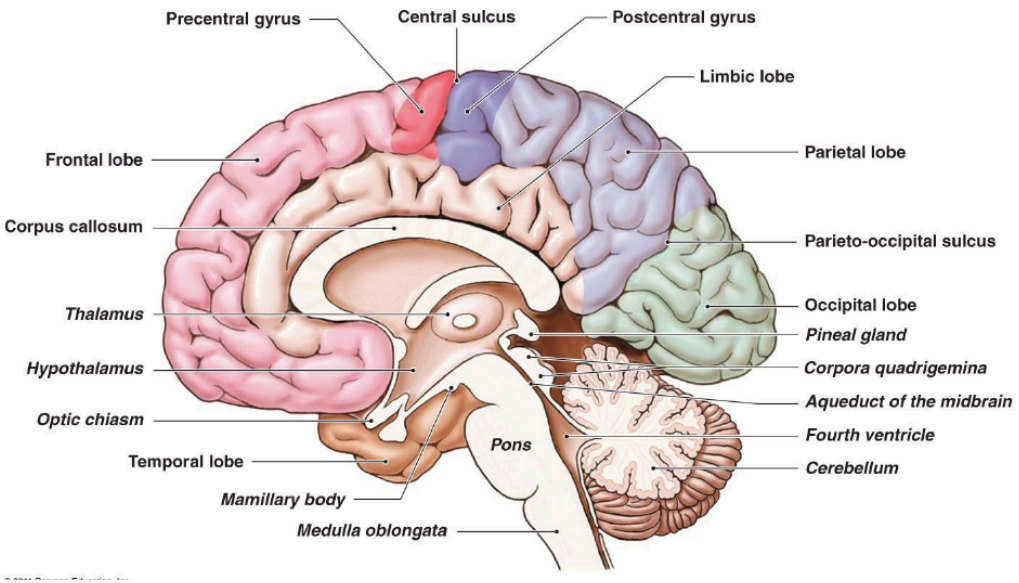
Training a Neural Network

Over the course of many **epochs** (one pass of training over the whole dataset), we'd expect to see our **loss steadily decrease**, which is indicative of our **predictions getting better**.



Typically we train on many images at the same time (which we call a **batch**), and **average our loss across them**.

Architecture of the Human Brain



If we share the same **brain architecture**,

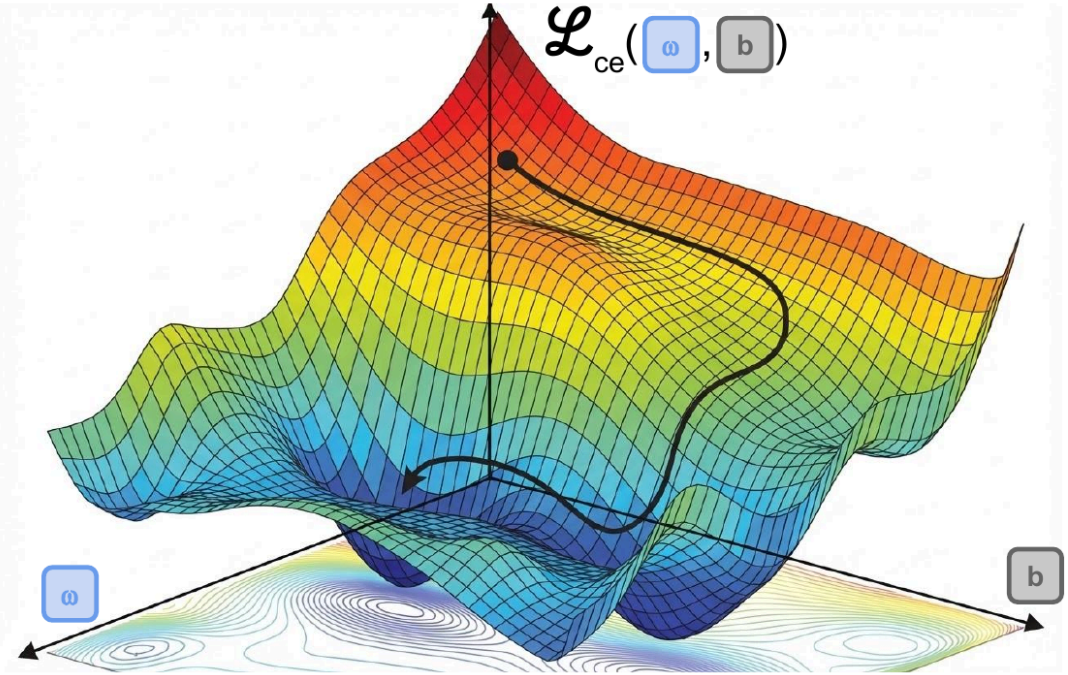
why are people so different?

Models as their Parameters

Much like we could imagine each possible `<28, 28>` input image being a `<1, 784>` dimensional vector in the same space, we can imagine **every possible model**, for this **architecture**, **being a single point** in a shared `<1, 7850>` dimensional space.

This means that **somewhere in this space is the best possible model**, or set of **parameters**, we could get. We just have to find that point!

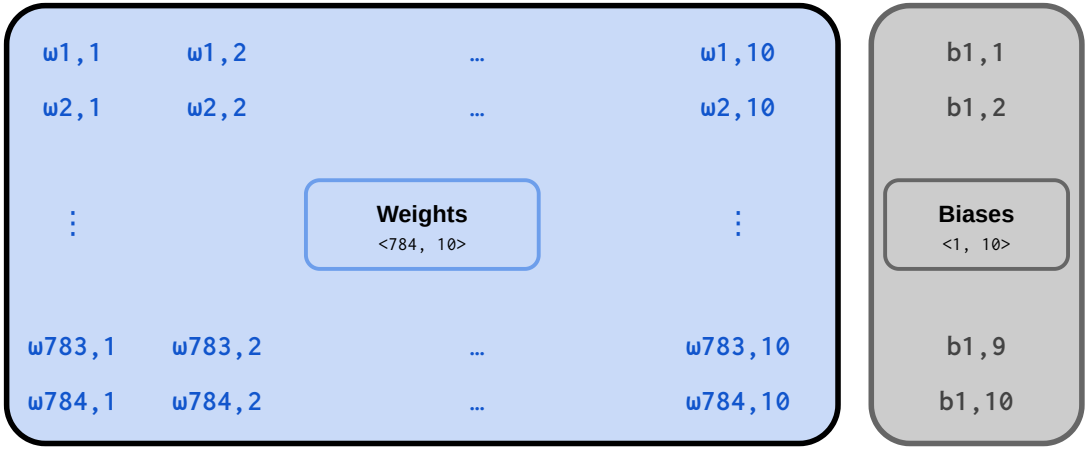
The Backwards Pass



$$\mathcal{L}_{ce}(W, b) = -\frac{1}{N}\left(\sum_{i=1}^{N} y_i \cdot \log(\text{softmax}(X W^T + b))\right)$$

Loss in terms of Parameters

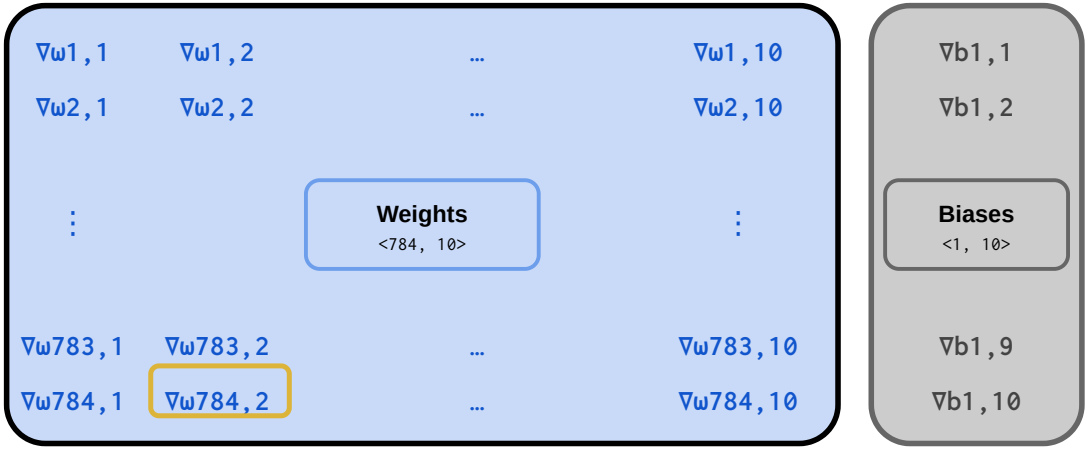
By writing our **loss** in terms of our **parameters**, and by **holding the training set constant**, we can determine how all **7850** parameters affected the loss calculation.



$$\mathcal{L}_{ce} = -\frac{1}{N}\left(\sum_{i=1}^{N} y_i \cdot \log(\text{softmax}(X W^T + b))\right)$$

Backpropagation

We can then partially differentiate the **loss** with respect to each of our **7850 parameters** to calculate what **adjustments we should make to each of them individually** in order to **reduce our loss**. We call these the **gradients**.



$$\frac{\partial \mathcal{L}_{ce}}{\partial w_{784,2}}\left(-\frac{1}{N}\sum_{i=1}^{N} y_i \cdot \log(\text{softmax}(X W^T + b))\right)$$

Backpropagation (Chain Rule)

the chain rule

$$\frac{\partial \mathcal{L}_{ce}}{\partial w_{784,2}} = \frac{\partial \mathcal{L}_{ce}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_{784,2}}$$

the derivative of $-(1/N)\ \Sigma\ y \cdot \log(\text{softmax}(z))$

$$\frac{\partial \mathcal{L}_{ce}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} = \frac{1}{N}(\hat{y} - y)$$

since $z = Xw^T + b$

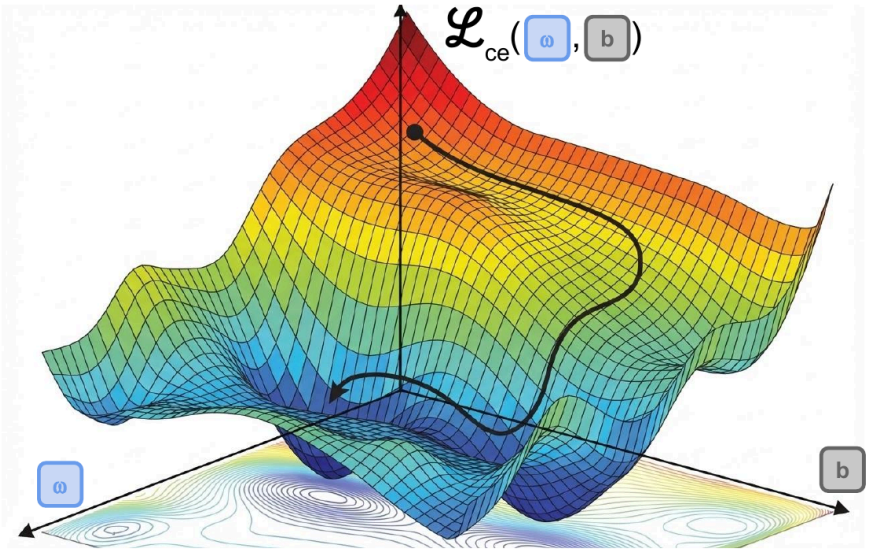
$$\frac{\partial z}{\partial w_{784,2}} = x^T$$

multiply the links together

$$\nabla w_{784,2} = \frac{1}{N} x^T (\hat{y} - y)$$

Gradient Descent

We update each **parameter** by subtracting from that parameter the value of the **gradient** multiplied by the **learning rate** (α). This should move us "downhill", or in the direction of **decreasing loss**.



$$\omega_{784,2} = \omega_{784,2} - \alpha \nabla \omega_{784,2}$$

$$\omega_{784,2} = \omega_{784,2} - \alpha \frac{1}{N} x^{T}(\hat{y} - y)$$

Backpropagation (Multilayer)

the forward pass: z_1, then ReLU, then z_2

$$z_1 = X W_1^T + b_1 \qquad A_1 = \mathrm{ReLU}(z_1) \qquad z_2 = A_1 W_2^T + b_2$$

two layers with a ReLU between, so a longer chain

$$\mathcal{L}_{ce} = -\frac{1}{N}\left(\sum_{i=1}^{N} y_i \cdot \log(\mathrm{softmax}(\mathrm{ReLU}(X W_1^T + b_1) W_2^T + b_2)) \right)$$

one link per operation

$$\frac{\partial \mathcal{L}_{ce}}{\partial w_{784,2}} = \frac{\partial \mathcal{L}_{ce}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_2} \cdot \frac{\partial z_2}{\partial A_1} \cdot \frac{\partial A_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_{784,2}}$$

the same softmax link, back through the second layer's weights, through ReLU's slope (1 where $z_1 > 0$, else 0), to the input

$$\frac{\partial \mathcal{L}_{ce}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_2} = \frac{1}{N}(\hat{y} - y) \qquad \frac{\partial z_2}{\partial A_1} = w_2 \qquad \frac{\partial A_1}{\partial z_1} = \mathrm{ReLU}'(z_1) \qquad \frac{\partial z_1}{\partial w_{784,2}} = x^T$$

multiply the links together: <784, 64> @ ((<64, 10> @ <10, 256>) times <64, 256> entry by entry), column 2

$$\nabla w_{1:784,2} = \frac{1}{N} x^T \left(\left[(\hat{y} - y) w_2 \right] \odot \mathrm{ReLU}'(z_1) \right)$$

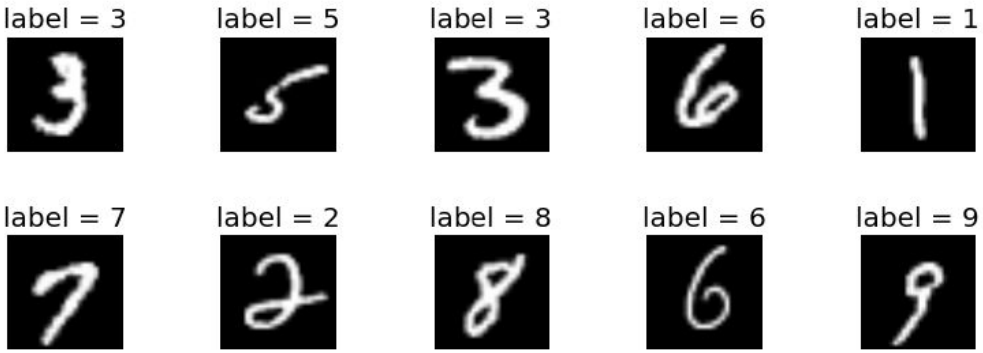
Programming: Google Colab

`Google Colab` is a service that provides hosted runtimes for `python notebooks`.

`python notebooks` are files that allow you to **mix text and code together** in a single document, and are one of the most common ways that **AI research and programming** is done.

Learning: Supervised, Task: Classification

We call it **supervised learning** when we have examples of the **target** in our **dataset**, because **we can tell our model if it guessed correctly or not**. We call these correct answers **ground truth**.



In **machine learning**, a **classification task** is one in which we're trying to **predict a category or label** (**target**) given a **sample**.

Representing an Output Prediction and the Ground Truth

We have **10** possible outputs (**targets/classes**) that we can produce, one for each number.

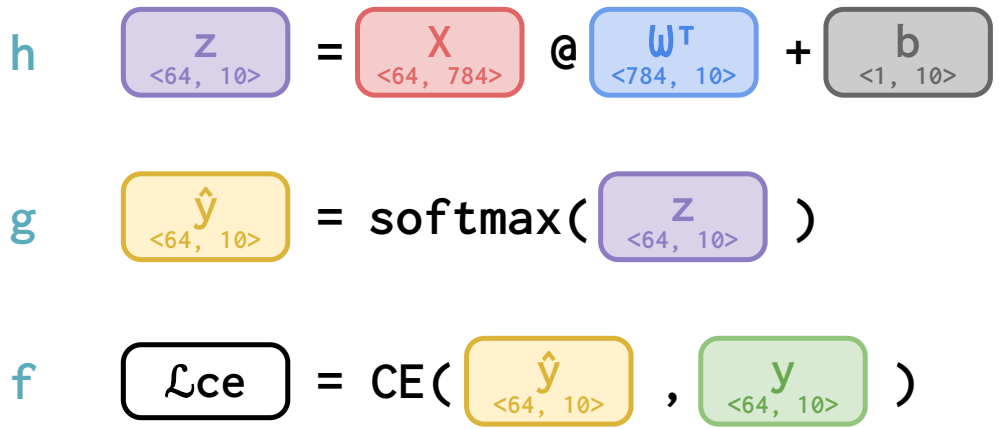


	0	1	2	3	4	5	6	7	8	9
Output ŷ: <1, 10>	0.0864	0.0956	0.1103	0.1796	0.0864	0.0703	0.0948	0.0925	0.0838	0.1003
Ground Truth y: <1, 10>	0.0	0.0	0.0	0.0	0.0	**1.0**	0.0	0.0	0.0	0.0

We return the **largest index value** as our prediction (**3** in this case), which we refer to as **ŷ**. Our **ground truth vector** is a **one-hot vector** with the **same size** as the output vector, with a **1** in the correct index and **0**s everywhere else. We refer to this vector as **y**.

Computation Graph

We can write our **neural network** as a **series of composite functions**, or a **computation graph**. By writing our **loss** in terms of our **parameters**, and by **holding the training set constant**, we can determine how all **7850** parameters affected the loss calculation.



Neural Networks as Composite Functions

$$\mathcal{L}_{ce} = -\frac{1}{N}\left(\sum_{i=1}^{N} y_i \cdot \log(\text{softmax}(X W^T + b))\right)$$

$$\mathcal{L}_{ce} = f(g(h(X, W, b)), y)$$

z: <64, 10>

$$h(X, W, b): \quad z = X W^T + b$$

ŷ: <64, 10>

$$g(z): \quad \hat{y} = \text{softmax}(z)$$

y: <64, 10>

$$f(\hat{y}, y): \quad -\frac{1}{N}\sum_{i=1}^{N} y_i \cdot \log(\hat{y}_i)$$

Computation Graph for the Backwards Pass

To actually determine how every **parameter** affects our loss, we can calculate their **gradients**, or derivatives, with respect to the **loss**. This will allow us to determine which direction we should adjust them (up or down) in order to **head in the direction of decreasing loss**!

Because a **neural network** is just a series of nested (composite) functions, we can do this using the **chain rule**.

$$\mathcal{L}_{ce} = f(g(h(X, W, b)), y)$$

$$\nabla W = \frac{\partial \mathcal{L}_{ce}}{\partial W} = \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial h} \cdot \frac{\partial h}{\partial W}$$

Updating our Parameters

the chain rule

$$\nabla \mathbf{w} = \frac{\partial \mathcal{L}_{ce}}{\partial \mathbf{w}} = \frac{\partial \mathcal{L}_{ce}}{\partial \hat{\mathbf{y}}} \cdot \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{z}} \cdot \frac{\partial \mathbf{z}}{\partial \mathbf{w}}$$

the derivative of $-(1/N)\ \Sigma\ y \cdot \log(\text{softmax}(z))$

$$\frac{\partial \mathcal{L}_{ce}}{\partial \hat{\mathbf{y}}} \cdot \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{z}} = \frac{1}{N}(\hat{\mathbf{y}} - \mathbf{y})$$

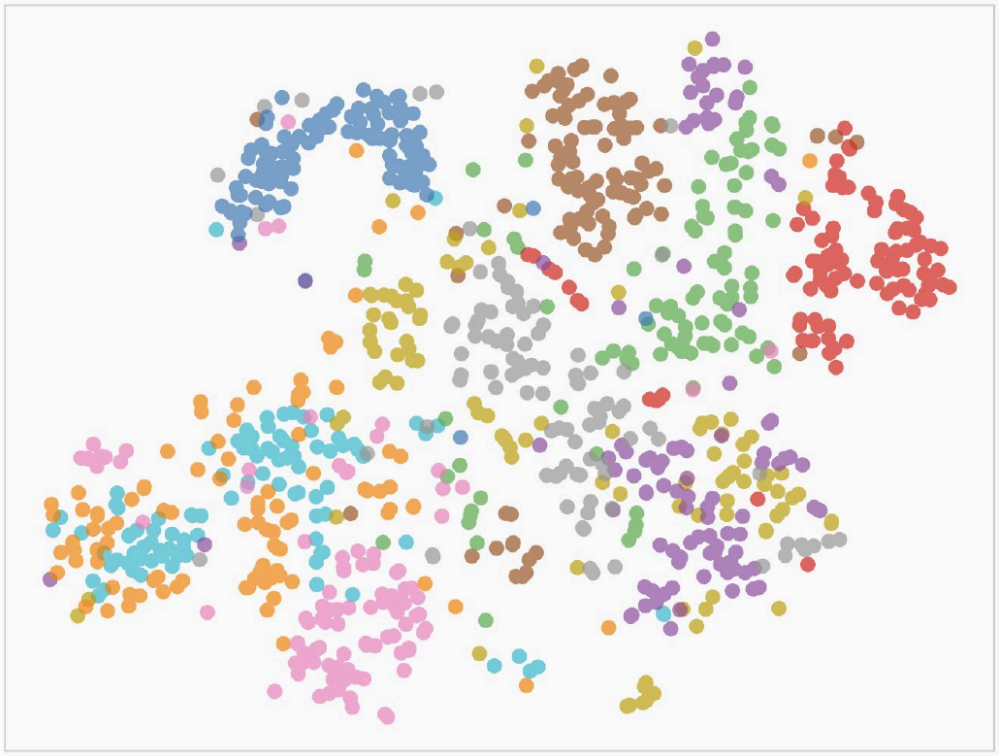
$z = XW^T + b$

$$\frac{\partial \mathbf{z}}{\partial \mathbf{w}} = \mathbf{x}^T$$

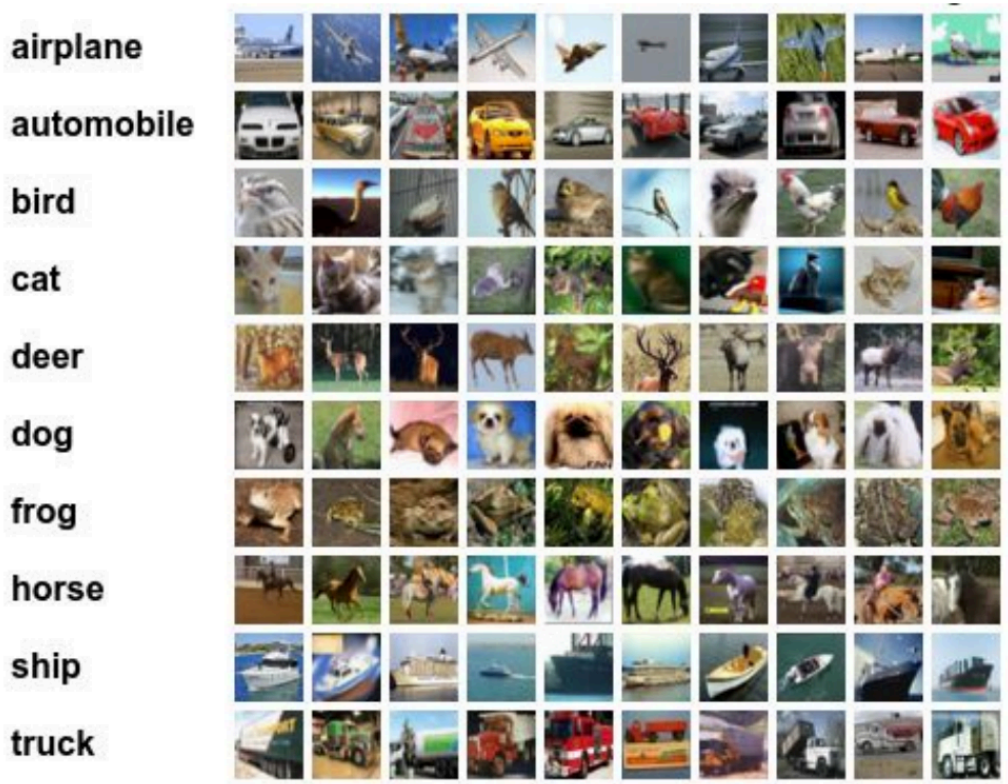
multiply the links together

$$\nabla \mathbf{w} = \frac{1}{N} \mathbf{x}^T (\hat{\mathbf{y}} - \mathbf{y})$$

Embeddings over Training

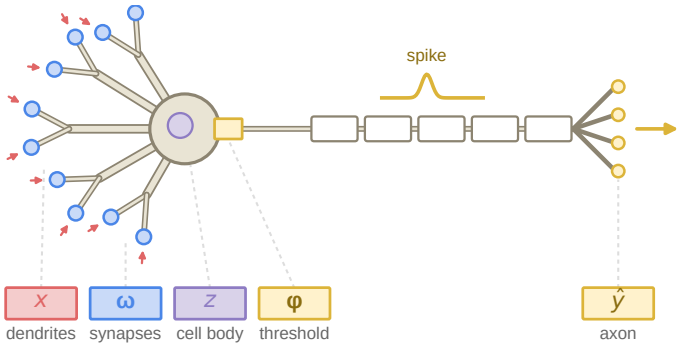


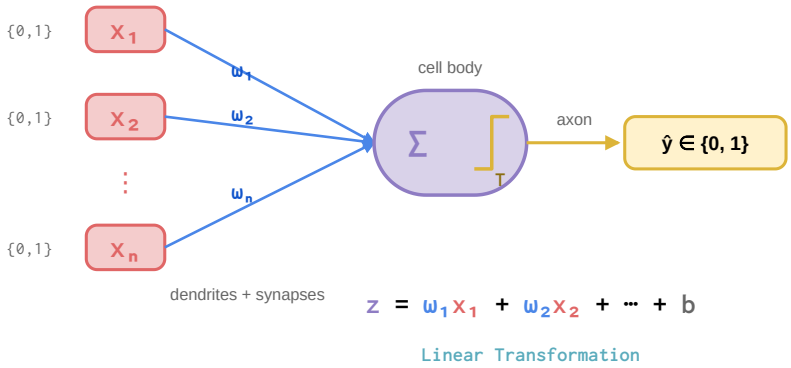
Classification Dataset: CIFAR-10



Activation Functions

Activation Function: a function applied to a neuron's weighted sum that determines the neuron's output.



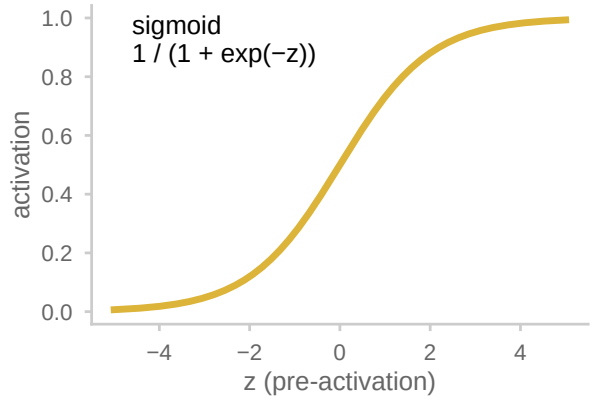


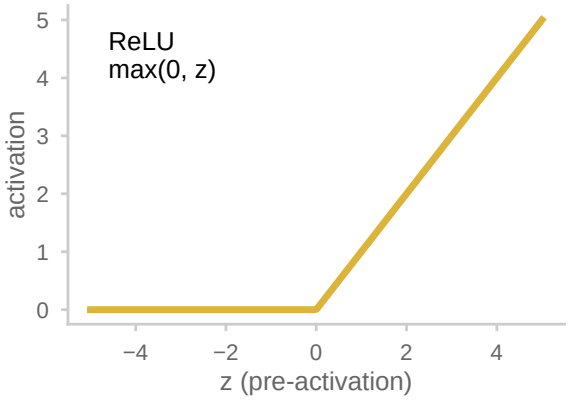
Activation Functions

Technically we've been using an **activation function** already, **softmax**, but for our **neural network** to actually be able to solve **nonlinear** problems, we need to add more of them! `MNIST` is actually just a very easy problem to solve.

Sigmoid: an S-shaped activation function that maps any real number to a value between `0` and `1`. **Softmax** can be thought of as a multi-class sigmoid.

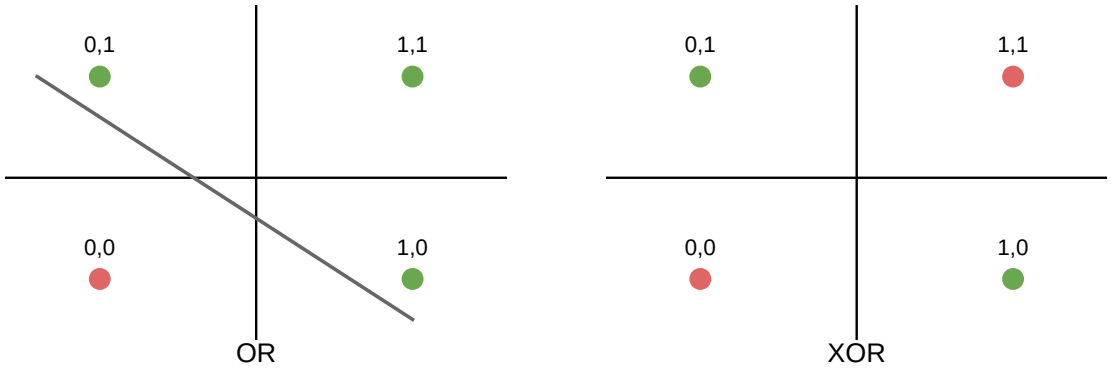
ReLU: the Rectified Linear Unit, which returns a `0` for any value less than `0` and the value itself for any value greater than `0`.





Linear and Nonlinear Problems

A **linear problem** is one which can be solved using a straight line (or hyperplane) while a **nonlinear problem** could not be.



This was famously demonstrated with the **XOR problem** in **"Perceptrons"** by Minsky and Papert, partially leading to the first AI winter.

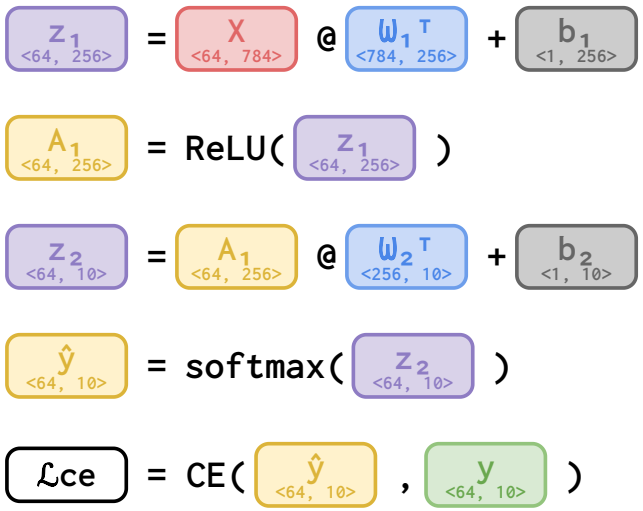
Computation Graph with Activation Functions

Essentially, we just add an **activation function after every linear layer**.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class FFN(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(28 * 28, 256)
        self.fc2 = nn.Linear(256, 10)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        logits = self.fc2(x)
        return logits
```



Next Class: A Choice

1. Another "wrap-up" lecture on **neural networks**
2. Start talking about using **text** as input to a **neural network**